Python AI Console Chat

Copilot Says: Perfect for AI-901/AI-102 labs. I'd put this at the top of the handout.

Python Setup Validation Commands

1. Check Python Version

Windows / Mac / Linux

python --version

Expected:

Python 3.12.x

If not found:

python3 --version

---

2. Verify Python Location

Windows

where python

Example:

C:\Users\Tim\AppData\Local\Programs\Python\Python312\python.exe

Mac / Linux

which python3

Example:

/usr/local/bin/python3

---

3. Verify Pip

pip --version

Expected:

pip 24.x from ...

---

4. Verify Scripts Path

Windows

echo %PATH%

Should contain something similar to:

Python312
Python312\Scripts

Common example:

C:\Users<user>\AppData\Local\Programs\Python\Python312
C:\Users<user>\AppData\Local\Programs\Python\Python312\Scripts

---

5. Verify Package Installation Works

pip install requests

Expected:

Successfully installed

---

6. Verify Python Runs

Create:

print("Python works")

Run:

python test.py

Expected:

Python works

---

7. Verify Virtual Environment (Recommended)

Create:

python -m venv .venv

Activate:

Windows

.venv\Scripts\activate

PowerShell

..venv\Scripts\Activate.ps1

Expected:

(.venv)

before the prompt.

AI Foundry Lab Connectivity Test

Minimal Inputs Required

Project Endpoint
API Key

Model Deployment Name

Examples:

Project Endpoint:
https://<project>.services.ai.azure.com/api/projects/<project>
Deployment:
gpt-5-mini
Key:
****

Simplest Console Chat Architecture

User
↓
Python
↓
Foundry Project
↓
gpt-5-mini
↓
Response

No GUI.

No Agent.

No MCP.

Just:

Question
↓
Answer

Student Flow

Enter Project Endpoint
↓

Enter API Key
↓
Enter Deployment Name
↓
Connect
↓
Success
↓
Prompt
↓
Response
↓
Prompt
↓
Response

"Does It Work?" Checklist

✓ python --version
✓ where python
✓ pip --version
✓ venv activates
✓ package install works
✓ endpoint copied
✓ deployment copied
✓ api key copied
✓ first model response returned

Foundry Learning Progression

Phase 1
Browser URL
(Use AI)
Phase 2
Python + Model
(Talk to AI)
Phase 3
Agent
(Workflow AI)

Phase 4
MCP
(AI Uses Tools)
Phase 5
Custom UI
(Users Consume AI)

For the labs, I'd start students with exactly this:

1. Install Python
2. Verify Python
3. Verify Pip
4. Enter Endpoint
5. Enter Key
6. Enter Deployment Name
7. Chat

A 30-line console application is enough to teach 80% of the Foundry model deployment concepts before introducing Agents, MCP, Search, or Voice Live.

This is about as small as I would make it while still being useful for AI-901 labs.

Save as:

foundry_chat.py

```
from openai import OpenAI
# Get connection info
endpoint = input("Project Endpoint: ").strip()
api_key = input("API Key: ").strip()
deployment = input("Model Deployment Name: ").strip()
# Build client
client = OpenAI(
    api_key=api_key,
    base_url=f"{endpoint}/openai/v1/"
)
print("\nConnected.")
print("Type 'exit' to quit.\n")
# Chat loop
while True:
    prompt = input("You: ")
    if prompt.lower() in ["exit", "quit"]:
        break
    response = client.responses.create(
```

```
    model=deployment,
    input=prompt
)
print("\nAI:")
print(response.output_text)
print()
```

Install Required Package

pip install openai

Run

python foundry_chat.py

Inputs

Project Endpoint:
https://your-project.services.ai.azure.com/api/projects/your-project
API Key:
xxxxxxxxxxxxxxxxxxxx
Model Deployment Name:
gpt-5-mini

Expected Flow

Connected.
You: What is Azure AI Foundry?
AI: Azure AI Foundry is...
You: Explain MCP.
AI: MCP stands for...
You: exit

Slightly Better Version (Recommended)

This version catches common connection mistakes.

```
from openai import OpenAI
endpoint = input("Project Endpoint: ").strip().rstrip("/")
api_key = input("API Key: ").strip()
deployment = input("Model Deployment Name: ").strip()
```

```
try:
    client = OpenAI(
        api_key=api_key,
        base_url=f"{endpoint}/openai/v1/"
    )
    print("\nConnection ready.")
    print("Type 'exit' to quit.\n")
    while True:
        prompt = input("You: ")
        if prompt.lower() in ["exit", "quit"]:
            print("Goodbye.")
            break
        try:
            response = client.responses.create(
                model=deployment,
                input=prompt
            )
            print("\nAI:")
            print(response.output_text)
            print()
        except Exception as ex:
            print(f"\nRequest failed: {ex}\n")
except Exception as ex:
    print(f"\nConnection failed: {ex}")
```

AI-901 Learning Progression

Browser URL
↓
foundry_chat.py
↓
Add Conversation History
↓
Add Search
↓
Add Agent
↓
Add MCP Tools
↓
Custom UI

This script gives students the absolute minimum:

Endpoint
+
Key
+
Deployment
+
Prompt
=
Response

with no agents, memory, MCP, search, voice, threads, or UI getting in the way of understanding a model deployment.